



UNIVERSIDAD TÉCNICA ESTATAL DE QUEVEDO

Facultad de Ciencias de la Computación

Análisis Crítico Comparativo de Apache Cassandra

Aplicando los Conceptos de Sistemas Distribuidos — Unidad 1

Estudiante: Emanuel Pino Juliana

Materia: Aplicaciones Distribuidas

Docente: Ing. Guerrero Ulloa Gleiston

Fecha: 20 Mayo 2026

Índice

1. Introducción	2
2. Fundamentos de Sistemas Distribuidos Aplicados a Cassandra	2
2.1. Definición y características	2
2.2. Historia y evolución	3
2.3. Comparación con sistemas centralizados y paralelos	3
3. Transparencias de Distribución	4
3.1. Transparencia de acceso	4
3.2. Transparencia de ubicación	4
3.3. Transparencia de replicación	4
3.4. Transparencia de fallos	5
4. Modelos de Comunicación	5
4.1. Modelo peer-to-peer e híbrido	5
4.2. Sockets y protocolo de transporte	5
4.3. RPC y su equivalente en Cassandra	5
4.4. Protocolo Gossip	6
4.5. Middleware en Cassandra	6
5. Sincronización y Relojes Distribuidos	6
5.1. El problema del tiempo en sistemas distribuidos	6
5.2. Algoritmo de Lamport	6
5.3. Cassandra y los timestamps	7
5.4. Mejoras: relojes híbridos lógicos	7
6. Modelo CAP y Tolerancia a Fallos	7
6.1. El teorema CAP	7
6.2. Posición de Cassandra: AP con consistencia ajustable	7
6.3. Mecanismos de tolerancia a fallos	8
7. Análisis Crítico	8
7.1. Fortalezas	8
7.2. Limitaciones y retos	8
7.3. Escenarios idóneos y no idóneos	9
8. Conclusiones	9

1. Introducción

Los sistemas distribuidos constituyen la base tecnológica de la infraestructura digital moderna. Desde plataformas de streaming hasta redes sociales con miles de millones de usuarios, la capacidad de distribuir el procesamiento y el almacenamiento entre múltiples nodos interconectados es hoy un requisito fundamental de la ingeniería de software [1].

Apache Cassandra es un sistema de gestión de bases de datos distribuido de código abierto, concebido originalmente por Facebook en 2007 para dar soporte a su función de búsqueda en la bandeja de entrada, y donado a la Apache Software Foundation en 2008 [2]. Su diseño incorpora principios del sistema Dynamo de Amazon y del modelo de datos Bigtable de Google, lo que lo convierte en un caso de estudio excepcional para analizar los fundamentos teóricos de la Unidad 1.

El presente trabajo aplica los siguientes ejes conceptuales del sílabo al caso de Apache Cassandra:

1. Definición, características y propiedades de los sistemas distribuidos.
2. Historia, evolución y contexto de aparición de Cassandra.
3. Ventajas, desafíos y retos: transparencia, concurrencia, escalabilidad y tolerancia a fallos.
4. Comparación con sistemas centralizados y paralelos.
5. Modelos de comunicación: cliente-servidor, peer-to-peer e híbrido; sockets, RPC y RMI.
6. Middleware y su rol en la distribución.
7. Sincronización y relojes distribuidos (algoritmo de Lamport).
8. Modelo CAP y tolerancia a fallos.

2. Fundamentos de Sistemas Distribuidos Aplicados a Cassandra

2.1. Definición y características

Tanenbaum y Van Steen [1] definen un sistema distribuido como un conjunto de computadoras independientes que se presentan al usuario como un sistema único y coherente. Cassandra cumple esta definición de forma rigurosa: un clúster puede estar compuesto por cientos de nodos físicamente separados, pero la aplicación cliente interactúa con una única interfaz lógica mediante CQL (*Cassandra Query Language*).

Cuadro 1: Características de sistemas distribuidos en Apache Cassandra

Característica	Manifestación en Cassandra
Concurrencia	Múltiples nodos procesan lecturas/escrituras en paralelo
Sin reloj global	Usa timestamps locales con resolución de conflictos LWW
Fallos parciales	Un nodo puede caer sin afectar la disponibilidad del clúster
Heterogeneidad	Nodos con distinto hardware pueden coexistir en el clúster
Escalabilidad	Agregar nodos incrementa capacidad sin detener el servicio
Apertura	Protocolo nativo CQL y drivers oficiales para múltiples lenguajes

2.2. Historia y evolución

Cassandra nació en 2007 dentro de Facebook como respuesta a la incapacidad de los sistemas relacionales para escalar a los volúmenes de datos de la red social. El paper fundacional [2] fue presentado en SOSP 2009 y describió por primera vez su arquitectura *masterless*. En 2010 se convirtió en proyecto de primer nivel de la Apache Software Foundation. Desde entonces ha evolucionado:

- **Cassandra 1.x (2011–2012):** introducción de CQL como reemplazo de la API Thrift.
- **Cassandra 2.x (2013–2015):** soporte para *lightweight transactions* (Paxos).
- **Cassandra 3.x (2015–2019):** motor de almacenamiento completamente reescrito.
- **Cassandra 4.x (2021–presente):** mejoras en observabilidad y protocolo nativo v5 [3].

2.3. Comparación con sistemas centralizados y paralelos

Una base de datos centralizada almacena todos sus datos en un único servidor, creando un único punto de falla y limitando la escalabilidad al hardware disponible. Un sistema paralelo (MPP) distribuye el procesamiento pero mantiene un coordinador centralizado.

Cassandra supera ambos mediante su arquitectura *masterless*: todos los nodos son funcionalmente equivalentes y cualquiera puede actuar como punto de entrada [4].

Cuadro 2: Comparación entre modelos de sistema

Criterio	Centralizado	Paralelo (MPP)	Distribuido (Cassandra)
Punto de falla	Único	Coordinador	Ninguno
Escalabilidad	Vertical	Limitada	Horizontal lineal
Consistencia	Fuerte (ACID)	Fuerte	Eventual/ajustable
Disponibilidad	Baja ante fallo	Media	Alta
Complejidad op.	Baja	Media	Alta

3. Transparencias de Distribución

Las transparencias de distribución describen en qué medida un sistema oculta su naturaleza distribuida. Tanenbaum y Van Steen [1] identifican ocho tipos principales.

3.1. Transparencia de acceso

Cassandra expone una única interfaz CQL sin importar si los datos residen local o remotamente. El driver cliente envía la solicitud al *coordinator node*, que localiza y retorna los datos de forma transparente para la aplicación.

3.2. Transparencia de ubicación

Los clientes se conectan mediante nodos semilla. El protocolo *gossip* permite al driver descubrir automáticamente todos los nodos del clúster; la posición física de un dato es opaca para la aplicación.

3.3. Transparencia de replicación

El factor de replicación se configura a nivel de *keyspace*. Cassandra replica automáticamente cada partición en N nodos sin que la aplicación deba conocer cuántas copias existen ni dónde.

3.4. Transparencia de fallos

Mediante *hinted handoff*, el coordinador almacena escrituras destinadas a nodos caídos y las reenvía al recuperarse. Para lecturas, cualquier réplica disponible puede servir la respuesta.

Cuadro 3: Nivel de transparencia de distribución en Cassandra

Tipo	Nivel	Observación
Acceso	Alto	CQL unifica acceso local y remoto
Ubicación	Alto	Gossip oculta la topología del clúster
Replicación	Alto	Factor configurable, invisible al cliente
Fallos	Medio	Hinted handoff cubre fallos temporales
Concurrencia	Medio	LWW puede causar conflictos silenciosos
Migración	Bajo	Rebalanceo requiere intervención operacional

4. Modelos de Comunicación

4.1. Modelo peer-to-peer e híbrido

Cassandra adopta un modelo **peer-to-peer (P2P)** entre nodos internos: todos son iguales, ninguno actúa como maestro permanente. Sin embargo, implementa un modelo **híbrido** en su arquitectura completa: la comunicación *cliente-a-clúster* sigue el modelo cliente-servidor, mientras que la comunicación *nodo-a-nodo* es puramente P2P [5].

4.2. Sockets y protocolo de transporte

La comunicación entre driver y Cassandra se realiza sobre **TCP/IP mediante sockets**. El Protocolo Nativo CQL (versiones 3 a 5) es un protocolo binario orientado a mensajes que opera sobre estos sockets, permitiendo multiplexar múltiples solicitudes simultáneas mediante identificadores de *stream* [6].

4.3. RPC y su equivalente en Cassandra

La llamada a procedimiento remoto (**RPC**) permite invocar funciones en procesos remotos como si fueran locales. Cassandra utilizaba originalmente **Apache Thrift**

(framework RPC) como interfaz principal, generando *stubs* para múltiples lenguajes mediante un IDL. A partir de Cassandra 4.x, Thrift fue eliminado en favor del Protocolo Nativo CQL. El concepto de RPC persiste en la comunicación interna entre nodos mediante *internode messaging* basado en Netty.

4.4. Protocolo Gossip

Para la coordinación interna, Cassandra utiliza el **protocolo gossip**: cada segundo, cada nodo intercambia información de estado con hasta tres vecinos aleatorios. La información se propaga en $O(\log N)$ rondas sin ningún componente centralizado [7].

4.5. Middleware en Cassandra

El **middleware** actúa como intermediario entre las aplicaciones y la infraestructura distribuida. En Cassandra se identifican dos niveles:

- **Driver de cliente** (ej. DataStax Java Driver): gestiona el pool de conexiones, balanceo de carga, política de reintentos y negociación del nivel de consistencia.
- **Plataformas de integración**: herramientas como Apache Kafka o Spark actúan como middleware entre Cassandra y otras fuentes de datos, habilitando pipelines de procesamiento en tiempo real.

5. Sincronización y Relojes Distribuidos

5.1. El problema del tiempo en sistemas distribuidos

En un sistema distribuido no existe un reloj global. Cada nodo tiene su propio reloj físico y, aunque NTP los sincroniza, siempre existe **deriva de reloj** (*clock skew*) [8], con implicaciones críticas para cualquier sistema que ordene eventos por timestamp.

5.2. Algoritmo de Lamport

Lamport [9] propuso los **relojes lógicos**: contadores enteros que se incrementan con cada evento local y se actualizan al recibir mensajes, garantizando orden causal sin relojes físicos sincronizados:

$$C(e) = \max(C_{\text{local}}, C_{\text{recibido}}) + 1 \quad (1)$$

Esto garantiza que si el evento a causó el evento b , entonces $C(a) < C(b)$.

5.3. Cassandra y los timestamps

Cassandra **no implementa el algoritmo de Lamport** en su versión estándar. Utiliza **timestamps de reloj físico** (microsegundos Unix) con resolución de conflictos **Last-Write-Wins (LWW)**: se conserva la escritura con el timestamp más alto. Si dos nodos tienen relojes desincronizados, una escritura más antigua puede sobrescribir una más nueva desde la perspectiva causal real [8].

5.4. Mejoras: relojes híbridos lógicos

Cassandra 4.x introdujo soporte experimental para **Hybrid Logical Clocks (HLC)**, que combinan tiempo físico con un contador lógico para capturar parcialmente la causalidad, reduciendo conflictos por *clock skew* [3].

6. Modelo CAP y Tolerancia a Fallos

6.1. El teorema CAP

El teorema CAP, formulado por Brewer [10] y demostrado por Gilbert y Lynch [11], establece que ante una partición de red (P), un sistema distribuido debe elegir entre consistencia (C) y disponibilidad (A).

6.2. Posición de Cassandra: AP con consistencia ajustable

Cassandra es **AP por diseño**: ante una partición de red prefiere seguir respondiendo aunque algunas réplicas no estén sincronizadas. Su modelo de **consistencia ajustable** permite acercarse a CP con la fórmula:

$$R + W > N \tag{2}$$

donde R es el quórum de lectura, W el de escritura y N el factor de replicación. Al cumplirse esta condición se garantiza que al menos una réplica actualizada participa en cada lectura.

6.3. Mecanismos de tolerancia a fallos

Cuadro 4: Mecanismos de tolerancia a fallos en Cassandra

Mecanismo		Descripción
Replicación multi-nodo		N copias por partición, distribuidas por data-center
Phi Accrual Detector		Detección probabilística continua de nodos caídos
Hinted Handoff		Escrituras diferidas para nodos temporalmente caídos
Read Repair		Sincronización de réplicas durante lecturas
Reparación anti-entropía		Reconciliación periódica con árboles de Merkle
Commitlog (WAL)		Recuperación ante fallos abruptos de proceso

7. Análisis Crítico

7.1. Fortalezas

Escalabilidad lineal y arquitectura masterless. La eliminación del nodo maestro resuelve el cuello de botella clásico de los sistemas relacionales. Empresas como Netflix operan más de 2.500 nodos Cassandra procesando millones de operaciones por segundo [12].

Consistencia ajustable. Cada operación puede tener un nivel de consistencia diferente, adaptado a sus requisitos específicos, sin cambiar el esquema ni la arquitectura.

Modelo P2P robusto. El gossip garantiza diseminación de estado en $O(\log N)$ rondas sin componentes centralizados, haciendo el sistema resistente a la pérdida de cualquier nodo individual.

7.2. Limitaciones y retos

Riesgo de conflictos por clock skew. El mecanismo LWW basado en timestamps físicos es vulnerable a relojes desincronizados, pudiendo producir pérdida silenciosa de datos en aplicaciones donde la causalidad importa [8].

Modelo de datos inflexible para consultas ad-hoc. El esquema debe diseñarse en función de los patrones de consulta conocidos. Las consultas no planificadas requieren

índices secundarios costosos o tablas duplicadas.

Ausencia de transacciones ACID completas. Las *lightweight transactions* (Paxos) cubren casos simples pero tienen alta latencia y no soportan transacciones multi-fila.

7.3. Escenarios idóneos y no idóneos

Cuadro 5: Idoneidad de Cassandra según escenario de uso

Idóneo para	No idóneo para
Telemetría e IoT a gran escala	Transacciones financieras ACID
Feeds de actividad en redes sociales	Consultas ad-hoc complejas con joins
Series de tiempo (métricas, logs)	Modelos de datos altamente relacionales
Sistemas multi-región activo-activo	Prototipos pequeños (overhead operacional alto)

8. Conclusiones

Sobre fundamentos y comparación. Cassandra es un ejemplo paradigmático de sistema distribuido: cumple las propiedades de concurrencia, ausencia de reloj global y tolerancia a fallos parciales. Su arquitectura masterless representa una evolución sobre los sistemas centralizados y paralelos, eliminando el único punto de falla al precio de mayor complejidad operacional.

Sobre transparencias. El sistema logra alta transparencia de acceso, ubicación y replicación. La transparencia de migración es el área de mayor debilidad, requiriendo intervención operacional explícita.

Sobre modelos de comunicación y middleware. El modelo híbrido P2P (internodo) y cliente-servidor (cliente-clúster) demuestra que los paradigmas no son mutuamente excluyentes. La transición de Thrift (RPC) al Protocolo Nativo CQL refleja la evolución hacia protocolos más eficientes. El driver actúa como middleware esencial de transparencia.

Sobre sincronización. La ausencia de relojes lógicos tipo Lamport y la dependencia de timestamps físicos es la debilidad conceptual más relevante frente a los fundamentos teóricos. Los HLC de Cassandra 4.x representan un avance hacia la corrección causal.

Sobre CAP y tolerancia a fallos. La elección AP con consistencia ajustable posiciona a Cassandra como referencia para aplicaciones de alta disponibilidad a escala global,

siempre que los requisitos de consistencia sean compatibles con el modelo eventual.

Referencias

- [1] A. S. Tanenbaum and M. Van Steen, *Distributed Systems: Principles and Paradigms*. Pearson, 3 ed., 2017.
- [2] A. Lakshman and P. Malik, “Cassandra: A decentralized structured storage system,” *ACM SIGOPS Operating Systems Review*, vol. 44, no. 2, pp. 35–40, 2010.
- [3] Apache Software Foundation, “Apache cassandra documentation,” 2024. Accessed May 2026.
- [4] G. DeCandia, D. Hastorun, M. Jampani, G. Kakulapati, A. Lakshman, A. Pilchin, S. Sivasubramanian, P. Voshall, and W. Vogels, “Dynamo: Amazon’s highly available key-value store,” in *Proceedings of the 21st ACM Symposium on Operating Systems Principles*, pp. 205–220, ACM, 2007.
- [5] DataStax, “Cassandra architecture: Data distribution and replication,” 2024. Accessed May 2026.
- [6] Apache Software Foundation, “Native protocol specification v5,” 2023. Accessed May 2026.
- [7] Apache Software Foundation, “Gossip protocol in apache cassandra,” 2024. Accessed May 2026.
- [8] M. Kleppmann, *Designing Data-Intensive Applications*. O’Reilly Media, 2017.
- [9] L. Lamport, “Time, clocks, and the ordering of events in a distributed system,” *Communications of the ACM*, vol. 21, no. 7, pp. 558–565, 1978.
- [10] E. A. Brewer, “Towards robust distributed systems,” in *Proceedings of the 19th Annual ACM Symposium on Principles of Distributed Computing*, p. 7, ACM, 2000.
- [11] S. Gilbert and N. Lynch, “Brewer’s conjecture and the feasibility of consistent, available, partition-tolerant web services,” *ACM SIGACT News*, vol. 33, no. 2, pp. 51–59, 2002.
- [12] C. Kalantzis, “Cassandra at netflix: Tuning for high volume write workloads,” 2021. Netflix Tech Blog, Accessed May 2026.